

13 Algorithmen zur Erzeugung von Zufallszahlen

13.1 Begriff und Abgrenzung

13.2 Aufbau von Zufallszahlengeneratoren

13.3 Abgeleitete Generatoren

13.4 Test

13.1 Begriff und Abgrenzung

- Für die Lösung von statistischen Aufgabenstellungen etc. aber auch in der betrieblichen Praxis (z. B. für die Planung der Entnahme von Stichproben zur Qualitätssicherung von Produktionsprozessen), werden Zufallszahlen benötigt
- Wir beschäftigen uns damit, wie man mit **deterministischen Mitteln (Algorithmen)** „Zufall erzeugen“ kann
- Es versteht sich von selbst, dass es sich dabei nicht um „echten“ oder „reinen“ Zufall handelt, bei dem Ereignisse völlig unvorhersehbar (kausal nicht erklärbar) eintreten
- Es handelt sich dabei um eine spezielle Art von **berechnetem Zufall**, der sich in Form von Zahlen manifestiert. Man kann deshalb in diesem Zusammenhang auch von „**künstlichem**“, „**quasi**“ oder „**Pseudo-Zufall**“ sprechen

Begriff und Abgrenzung

- Wenn in einem bestimmten Anwendungsbereich, z. B. in der Simulation, Zufallszahlen benötigt werden, dann ist man meist nicht an einer einzigen Zufallszahl interessiert, sondern benötigt in der Regel eine (lange) **Folge von Zahlen**, die als zufällig gebildet erscheint
- Damit ist gemeint, dass das Bildungsgesetz für eine solche Zahlenfolge im Dunkeln bleibt und man daher nicht vorhersagen kann, durch welche Zahl das nächste Glied einer solchen Folge gebildet wird
- Die Elemente einer solchen Folge sollen aus einem bestimmten **Wertebereich** stammen und einer bestimmten (statistischen) **Verteilung** folgen, aber unabhängig voneinander sein
- Wir nennen eine solche Folge (Pseudo-) **Zufallszahlenfolge** und Algorithmen, die sie erzeugen **Zufallszahlengeneratoren**

Anwendungsgebiete

Zufallszahlenfolgen braucht man für viele Aufgaben

- Simulation natürlicher Vorgänge: um Phänomene mit zufälligem Verhalten darzustellen (z. B. kernphysikalische Prozesse, Verkehrsprobleme)
- Stichproben
- Monte-Carlo-Methoden
- Test von Algorithmen mit zufälligen Daten
- Programmierung von Spielen und Fragen der künstlichen Intelligenz
- Kryptographie
- Animation
- ...

Begriff

Begriff Zufallszahlengenerator

- Darunter verstehen wir einen Algorithmus mit Gedächtnis, der bei jedem Aufruf eine Zahl aus einem bestimmten Wertebereich liefert, die in so unübersichtlicher Weise von früheren Aufrufen abhängt, dass sie als zufällige Größe erscheint

`x := IntRand()` *integer random number*

Begriff Zufallszahl

- Zahl aus einem bestimmten Wertebereich
- deren Wert regellos (= nicht vorhersagbar) ist
- die Teil einer Zahlenfolge ist, die eine bestimmte statistische Verteilung (z. B. Gleichverteilung) aufweist und die bestimmte Tests (z.B. Chi²-Test) erfüllt

Beispiel: Würfel liefert gleichverteilt Zahlen aus dem Bereich 1 .. 6



Begriff Zufallszahlengeneratoren

- Algorithmen mit denen man Zufallszahlenfolgen generieren kann

```
var  
  x: int
```

```
x := IntRand()
```

Generator für ganzzahlige Werte $0..2^{16}-1$



- Typischer Zahlenbereich $0..m-1$ mit $m = 2^k$ und $k = 16, 32, \dots$
- Widerspruch: Zufall und Algorithmen
 - Algorithmusbegriff erfordert Eindeutigkeit (Vorhersagbarkeit)
 - Zufallszahlen sollen aber nicht vorhersagbar sein
- Abhilfe: Zufallszahlen werden nach einer Regel erzeugt, die „undurchschaubar“ ist (Pseudozufallszahlen)

Generierungsprinzip

Die (jeweils nächste generierte) Zufallszahl hängt von der Vorgeschichte ab

Aufruf	Zufallszahl	Zufallszahl (einfach)
1	k_1	k_1
2	$k_2 = f(k_1)$	$k_2 = f(k_1)$
3	$k_3 = f(k_1, k_2)$	$k_3 = f(k_2)$
$\vdots$	$\vdots$	$\vdots$
n	$k_n = f(k_1, k_2, \dots, k_{n-1})$	$k_n = f(k_{n-1})$

- Zufallszahlengeneratoren sind **Algorithmen mit Gedächtnis**
- Pseudozufallszahlen, weil die **Zufallszahlenfolge reproduzierbar** ist, wenn man den **Startzustand** kennt

13.2 Aufbau von Zufallszahlengeneratoren

- Es ist durchaus nicht selbstverständlich, dass es überhaupt Methoden gibt, mit denen man auf **arithmetischem Wege regellose Zahlenfolgen**, die also dem Betrachter als Zufallszahlenfolge erscheinen, erzeugen kann
- John von Neumann erfand 1946 die sogenannte „Quadrierungsmethode“, bei der eine gegebene Zahl x , bestehend aus n Bit, quadriert und von dem $2n$ Bit langen Ergebnis die mittleren n Bit als Zufallszahl genommen wird

Beispiel

- Startet man z. B. mit $x = 42$, so ergibt sich die Folge 42, 76, 77, 92, 46, ...

$42^2 = 1764$	$76^2 = 5776$	$77^2 = 5929$	$92^2 = 8464$	$46^2 = \dots$
---------------	---------------	---------------	---------------	----------------

13.2.1 Lineare-Kongruenz-Methode (LKM)

Standardverfahren nach Derrick H. Lehmer (1949)

$$x_{n+1} = (a \cdot x_n + c) \bmod m$$

mit

▪ Modul	m	möglichst groß
▪ Multiplikator	a	$2 \leq a < m$
▪ Inkrement	c	$0 \leq c < m$
▪ vorherige Zufallszahl	x_n	$0 \leq x_n < m$
▪ nächste Zufallszahl	x_{n+1}	$0 \leq x_{n+1} < m$

Güte des Generators hängt von Wahl der Faktoren a , c und m ab

Beispiele

Beispiel 1: LKM mit $a = 5$, $c = 3$, $m = 8$ und $x_0 = 0$

- $x_{n+1} = (5 \cdot x_n + 3) \bmod 8$
- liefert Zahlenfolge mit Periodenlänge 8:

3 2 5 4 7 6 1 0 3 2 5 4 7 6 1 0 3 2 ...

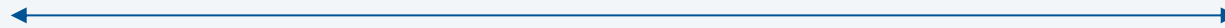


Periodenlänge = 8

Beispiel 2: LKM mit $a = 5$, $c = 1$, $m = 16$ und $x_0 = 0$

- $x_{n+1} = (5 \cdot x_n + 1) \bmod 16$
- liefert Zahlenfolge mit Periodenlänge 16:

1 6 15 12 13 2 11 8 9 14 7 4 5 10 3 0 1 6 ...



Periodenlänge = 16

Beide Generatoren erreichen die volle Periodenlänge $m = 8$ und $m = 16$

Beispiele

Beispiel 3: LKM mit $a = 3$, $c = 5$, $m = 8$ und $x_0 = 0$

- $x_{n+1} = (3 \cdot x_n + 5) \bmod 8$
- liefert Zahlenfolge mit Periodenlänge 4:

5 4 1 0 5 4 1 0 5 4 1 0 5 4 1 0 5 4 ...

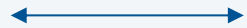


Periodenlänge = 4

Beispiel 4: LKM mit $a = 19$, $c = 1$, $m = 381$ und $x_0 = 0$

- $x_{n+1} = (19 \cdot x_n + 1) \bmod 381$
- liefert Zahlenfolge mit Periodenlänge 3:

1 20 0 1 20 0 1 20 0 1 20 0 1 20 0 1 20 0 ...



Periodenlänge = 3

Wunsch: Periodenlänge mindestens m

Regeln zur Wahl von m , a , c (nach Knuth und Sedgewick)

Modul m :

- möglichst groß
- typisch 2^k oder 2^{k-1} damit $x \bmod m$ einfach berechnet werden kann

Multiplikator a :

- um rund eine Zehnerpotenz kleiner als m , also $a \approx m/10$
- und von der Form $a = \dots g21$ mit gerader Ziffer g
(unregelmäßiges Bitmuster, z.B. 110101011101)

Inkrement c :

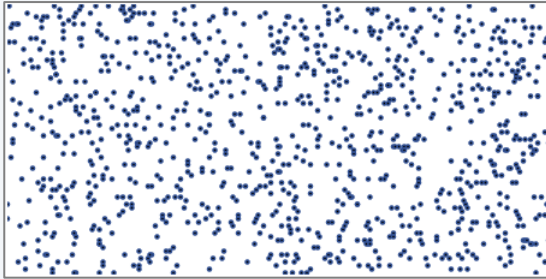
- $c = 1$

Startwert beliebig zwischen 0 und $m - 1$

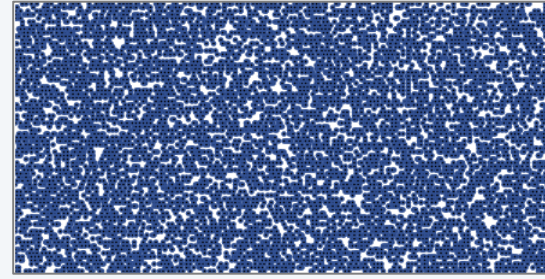
Gutes Beispiel: $a = 3421$, $c = 1$, $m = 2^{16}$ und $x_0 = 0$

Vergleich

- Nach Regeln von Knuth u. Sedgewick: $x_{n+1} = (3421 \cdot x_n + 1) \bmod 2^{16}$

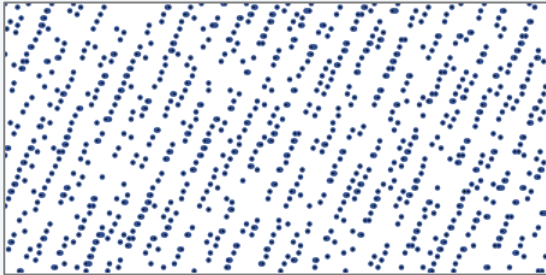


n = 1000

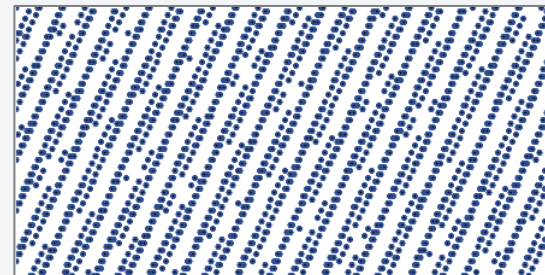


n = 10000

- Ändern von a von 342**1** auf 342**3**



n = 1000



n = 10000

- Muster (unten) sind Hinweis auf schlechte Generatoren

Standard *LKM*-Generator IntRand

Nach der Linearen-Kongruenz-Methode lässt sich der folgende (Standard-) Zufallszahlengenerator implementieren:

```
IntRand(): int
  const
    m = 65536
    a = 3421
    c = 1
  static var
    x: int := 0
begin
  x := (a*x + c) mod m
  return x
end IntRand
```

Zufälliger Startwert

- Startwert bestimmt Zufallsfolge
- wird meist *seed* (Saat, Keim) bezeichnet
- kann in der Regel mit einer eigenen Funktion (SetSeed) eingestellt werden

Beispiel: zur Erzeugung einer einfach zu rekonstruierenden Zufallszahlenfolge

```
SetSeed(↓42)
```

Beispiel: zur Erzeugung einer schwer (nicht) rekonstruierbaren Zufallszahlenfolge

```
SetSeed(↓GetTime())
```

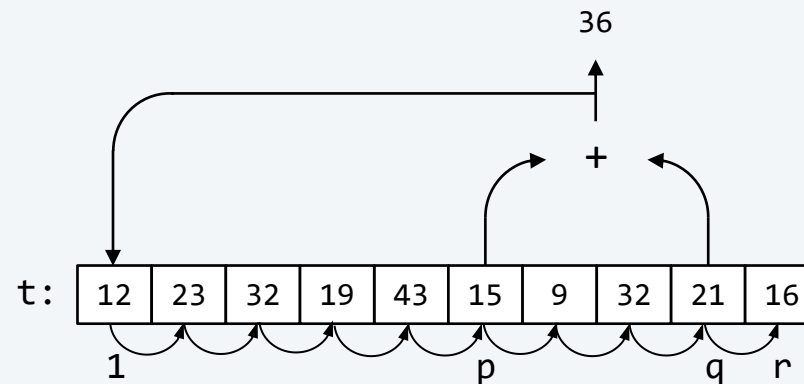
13.2.2 Methoden zur Verlängerung der Periodenlänge

- Für manche Anwendungen (z. B. für Simulationsaufgaben in der Physik) sind Zufallszahlenfolgen mit größeren Periodenlängen erforderlich, als sie die lineare Kongruenzmethode liefern kann
- Es existiert eine Vielzahl von Vorschlägen, um die Periodenlängen zu vergrößern
- Um zu illustrieren, wie man dabei vorgehen kann, greifen wir zwei solcher Vorschläge, die sich besonders bewährt haben, heraus:
 - die Schieberegistermethode von Tausworthe und
 - die Tabellenmethode von MacLaren und Marsaglia
- Durch diese beiden Methoden kann die Periodenlänge vorhandener Generatoren deutlich erhöht werden

Schieberegistermethode nach Tausworthe

Lösungsidee

- fülle ein Feld $t[1:r]$ mit Zufallszahlen (z. B. mit `IntRand`)
- verknüpfe zwei Elemente ($t[p]$ und $t[q]$) (z.B. mit Operator $+$)
- verschiebe den Feldinhalt um eine Stelle nach rechts
- verwende das Verknüpfungsergebnis als Zufallszahl und fülle $t[1]$ damit



Hinweise für Implementierung

- Aus Effizienzgründen wird nicht der Feldinhalt verschoben, sondern p , q und i werden verringert ($m = 45$)

t:	12	23	32	19	43	15	9	32	21	16
	i				p			q	r	

t:	36	23	32	19	43	15	9	32	21	16
	i				p			q	r	

$$36 = (15 + 21) \bmod 45$$

t:	36	23	32	19	43	15	9	32	21	30
				p			q			i

$$30 = (43 + 32) \bmod 45$$

t:	36	23	32	19	43	15	9	32	28	16
			p			q			i	r

$$28 = (19 + 9) \bmod 45$$

- Beispiele für Verknüpfungen
 - bitweises xor: $t[p] \text{ xor } t[q]$
 - Addition: $(t[p] + t[q]) \bmod m$
- Gute Ergebnisse mit $p = 31, q = r = 55$ (nach Knuth)

Algorithmus

```
const
  r = 55
var
  t: array [1:r] of int
  i, p, q: int
```

```
InitTausworthe()
```

```
begin
```

```
  for i := 1 to r do
```

```
    t[i] := IntRand()
```

```
  end -- for
```

```
  i := 1; p := 31; q := 55
```

```
end InitTausworthe
```

Initialisierung mit einem LKM-Generator



Initialisierung nach Knuth



```
Tausworthe(): int
```

```
begin
```

```
  i := i - 1; if i = 0 then i := r end
```

```
  p := p - 1; if p = 0 then p := r end
```

```
  q := q - 1; if q = 0 then q := r end
```

```
  t[i] := t[p] xor t[q]
```

```
  return t[i]
```

```
end Tausworthe
```

Tabellenmethode nach MacLaren und Marsaglia

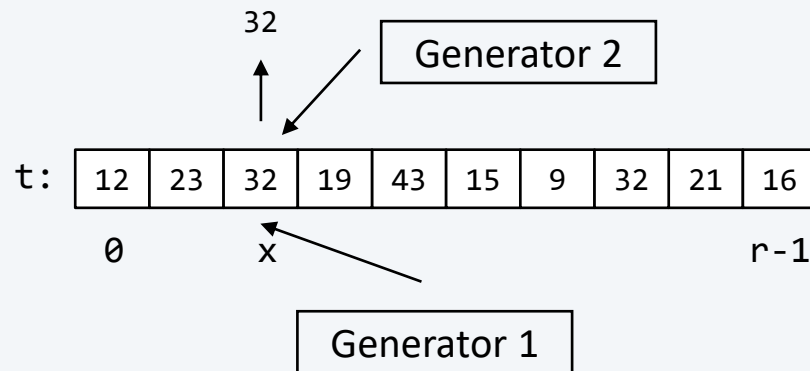
Lösungsidee

Initialisierung

- fülle Tabelle $t[0:r-1]$ mit Zufallszahlen erzeugt mit Generator 1

Generierung einer Zufallszahl

- erzeuge mit Generator 2 eine Zufallszahl x aus dem Intervall $0:r-1$
- verwende $t[x]$ als gesuchte Zufallszahl
- überschreibe $t[x]$ mit Zufallszahl erzeugt durch Generator 1



Algorithmus

```
const
  r = 100
var
  t: array [0:r-1] of int
```

```
InitMacLaren()
begin
  for i := 0 to r - 1 do
    t[i] := IntRand()
  end -- for
end InitMacLaren
```

```
MacLaren(): int
var
  x, pos: int
begin
  pos := RangeRand(↓r)
  x := t[pos]
  t[pos] := IntRand()
  return x
end MacLaren
```

Index aus [0 .. r-1]



13.3 Abgeleitete Generatoren

Aus Generatoren, die ganzzahlige Werte aus dem Intervall $[0:m)$ liefern, können Generatoren für beliebige Werte aus **anderen Intervallen** abgeleitet werden, z. B:

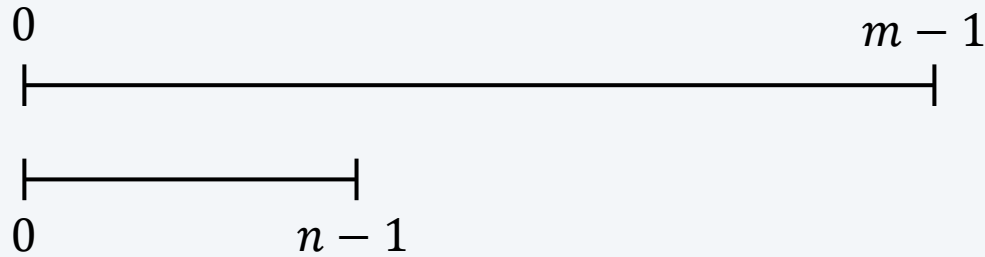
- `RealRand()`: `real`, Gleitkomma-Zufallszahlen aus dem Intervall $[0:1)$ (Problem: Genauigkeit)
- `RangeRand( $\downarrow n$ )`: `int`, ganzzahlige Zufallszahlen aus dem Intervall $[0:n)$ (Problem: Gleichverteilung)

Aus Generatoren, die gleichverteilte Zufallszahlenfolgen liefern, können Generatoren mit **anderen Verteilungen** abgeleitet werden, z. B.

- `RangeGaussRand( $\downarrow n$ )`: `int`, liefert normalverteilte Zufallszahlenfolgen

Generatoren für kleinere Wertebereiche

Abbildung des Bereichs $0..m - 1$ auf $0..n - 1$ mit $n < m$



Möglichkeiten:

- Modulo $z := \text{IntRand()} \bmod n$
- Skalierung $z := \text{IntRand()} * n \text{ div } m$

Hinweis:

Überlaufgefahr

- Skalierung ist besser, da Modulo-Operation niederwertige Bit-Stellen verwendet und sich dies bei LKM auf die Zufälligkeit nicht besonders günstig auswirkt

Generator für Gleitkomma-Zufallszahlen aus $[0:1)$

Neben ganzzahligen Zufallszahlenfolgen werden häufig reellwertige Zufallszahlen aus dem halboffenen Intervall $[0 : 1)$ benötigt

Der Algorithmus `RealRand` erzeugt, unter Verwendung eines Generators `IntRand`, der ganzzahlige Zufallszahlen aus dem Wertebereich $[0 : m-1]$ liefert, mittels Division dieser Zahlen durch m reellwertige Zufallszahlen aus im Bereich $[0 : 1)$

```
RealRand(): real
begin
  return Real( $\downarrow$ IntRand()) / Real( $\downarrow$ m)
end RealRand
```

Im Sinne der Mathematik umfasst das Intervall zwar unendlich viele reelle Zahlen, `RealRand` kann aber nur m davon liefern, weshalb m – wie schon erwähnt – möglichst groß sein sollte

Generatoren für intervallbezogene ganzzahlige Zufallszahlenfolgen

Oft benötigt man Zufallszahlenfolgen, die ganzzahlig sind und aus einem bestimmten Wertebereich (*range*) stammen, von einer Untergrenze $lb \geq 0$ bis zu einer Obergrenze $ub < m$

Beispiele dafür sind die Simulation eines Münzwurfs oder eines Würfels

Man kann solche Zufallszahlen auf einfache Weise z. B. über einen „Umweg“ aus reellwertigen Zufallszahlen erzeugen:

```
RangeRand(↓lb: int ↓ub: int): int
  var
    n: int
  begin
    n := ub - lb + 1
    return lb + Floor(↓RealRand() * Real(↓n))
  end RangeRand
```

Generatoren für intervallbezogene ganzzahlige Zufallszahlenfolgen

Eine andere Möglichkeit für die Erzeugung intervallbezogener ganzzahliger Zufallszahlenfolgen, ist es, IntRand zu verwenden, z. B.

```
RangeRand(↓lb: int ↓ub: int): int
  var
    n: int
  begin
    n := ub - lb + 1
    return lb + (IntRand() mod n)
  end RangeRand
```

Der Algorithmus RangeRand liefert zwar Zufallszahlen aus dem gewünschten Intervall $[lb : ub]$, diese sind u. U. aber nicht gleichverteilt, auch wenn IntRand, der Zufallszahlen aus $[0 : m - 1]$ liefert, diese Eigenschaft für seine Ergebnisse garantiert; nämlich, wenn n kein Teiler von m ist

Problem der Gleichverteilung

- zuerst muss die Schranke k berechnet werden, bis zu der hin die Zufallszahlen direkt in den kleineren Bereich transformiert werden können
- dann muss eine Zufallszahl aus dem großen Bereich ermittelt werden, die unter dieser Schranke liegt
- diese Zufallszahl darf dann mit der Modulo-Operation in das kleinere Intervall transformiert werden

Der Algorithmus RangeRand1 implementiert dieses Verfahren:

```
RangeRand1(↓lb: int ↓ub: int): int
  var n, k, ir: int
begin
  n := ub - lb + 1
  k := m div n;   k := k * n
  repeat
    ir := IntRand()
  until ir < k
  return lb + (ir mod n)
end RangeRand1
```

Normalverteilte Zufallszahlenfolgen

Neben gleichverteilten Zufallszahlenfolgen kommt normal- oder gaussverteilten Zufallszahlenfolgen eine wichtige Rolle zu

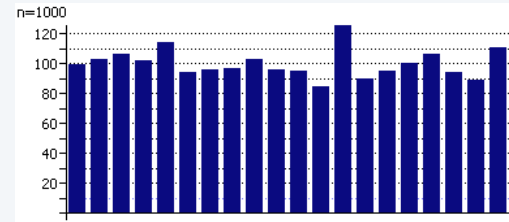
Viele Eigenschaften in der Praxis, z. B. der Durchschnitt einer großen Stichprobe, oder Phänomene in der Natur, z. B. die Körpergröße von Menschen, sind (annähernd) normalverteilt

Grenzwertsatz der Statistik: die standardisierte Summe von n unabhängigen Zufallsvariablen, die alle dieselbe Verteilung (z. B. die Gleichverteilung) aufweisen, konvergiert für n gegen unendlich zur **Normalverteilung**

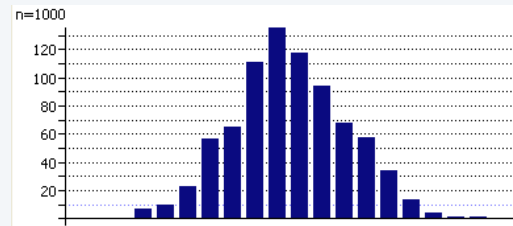
Daher lässt sich eine normalverteilte Zufallszahlenfolge durch einen Zufallszahlengenerator erzeugen, der eine gleichverteilte Zufallszahlenfolge als Basis verwendet, indem ein neues Element der normalverteilten Folge aus dem Durchschnitt der nächsten n Elemente der gleichverteilten Folge gebildet wird

Normalverteilte Zufallszahlenfolgen

```
RangeRandGauss(↓lb: int ↓ub: int): int
  const
    n = 8
  var
    sum, i: int
begin
  sum := 0
  for i := 1 to n do
    sum := sum + RangeRand(↓lb ↓ub)
  end -- for
  return sum div n
end RangeRandGauss
```



n typisch zwischen 6 und 10



13.4 Test zur Prüfung der Güte von Zufallszahlengeneratoren

Die Güte von Zufallszahlenfolgen kann mit einer Reihe von Tests überprüft werden

In den Arbeiten von Knuth und Sedgewick finden wir eine Vielzahl solcher Zufälligkeitstests mit all ihren mathematischen Einzelheiten

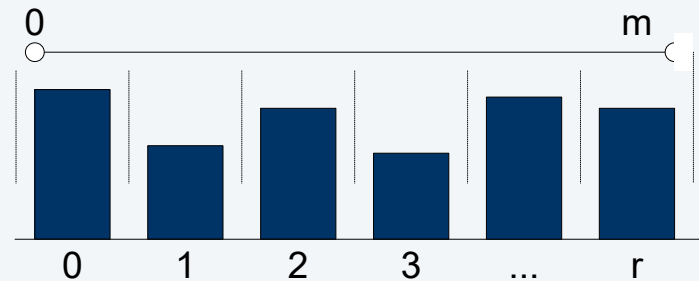
Wir beschränken uns darauf, einige einfache und für die Praxis wichtige Tests kurz aber so zu skizzieren, dass die Studierenden in der Lage sind, sie bei Bedarf anzuwenden

Test zur Prüfung der Güte von Zufallszahlengeneratoren

- **Häufigkeitstest**: Dieser Test dient dazu, die Auftrittshäufigkeiten der von einem Generator gelieferten Zufallszahlen zu ermitteln.
- **Serientest**: Dieser Test dient – im Unterschied zum Häufigkeitstest – dazu, die Auftrittshäufigkeit von Zahlenpaaren zu ermitteln.
- **Lückentest**: Der Lückentest untersucht, in welchen Abständen sich eine bestimmte Zufallszahl in der Folge wiederholt (für die lineare Kongruenzmethode ist das für alle Zahlen die Periodenlänge).
- **Läufetest**: Ein Lauf (*run*) ist eine Teilfolge von auf- oder absteigenden Elementen in einer größeren Folge. Der Läufetest untersucht, wie viele solcher auf- oder absteigende Teilfolgen (also Läufe) einer bestimmten Länge in der Zahlenfolge enthalten sind.
- **Chi²-Test** und **Himmelstest**: Zwei weitere, für den praktischen Einsatz wichtige Tests von Zufallszahlenfolgen, sind der sogenannte Chi²-Test und der sogenannte Himmelstest.

Häufigkeitstest

- Einteilung der Zufallszahlen in gleich große Klassen 1.. r
- n Zufallszahlen werden in r Klassen eingeteilt
- In jeder Klasse sollten n/r Werte sein



- Bei linearer Kongruenzmethode und $n = m$ wird jede Zahl 1x erzeugt, daher immer gleich verteilt
- Gleichverteilung sagt nichts über Serie aus, z.B. für $n = 5$

1 2 3 4 5 1 2 3 4 5 1 2 3 4 ...

1 1 2 2 3 3 4 4 5 5 1 1 2 2 ...

Serientest

- Untersuchung von aufeinander folgenden Zufallszahlen (Paare)
- Wahrscheinlichkeit dafür ist $p_s = 1/r$ (z. B. bei Würfeln $p_s = 1/6$)

Beispiel: Serientest: mit $r = 5$ sollte $p_s = 1/5$ sein

1 2 3 4 5 1 2 3 4 5 1 2 3 4 ...

$$p_s = 0$$

1 1 2 2 3 3 4 4 5 5 1 1 2 2 ...

$$p_s = 1/2$$

Beispiel Serientest: mit $r = 3$ sollte $p_s = 1/3$ sein

1 1 1 2 1 3 2 1 2 2 2 3 3 1 3 2 3 3 1 1 ...

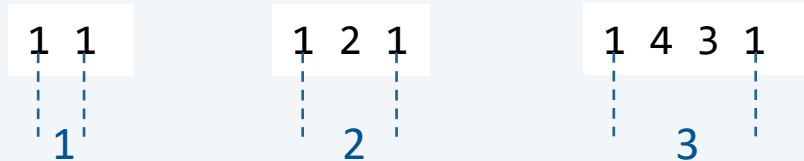
$$p_s = 1/3$$

... erfüllt sowohl den Gleichverteilungstest als auch den Serientest

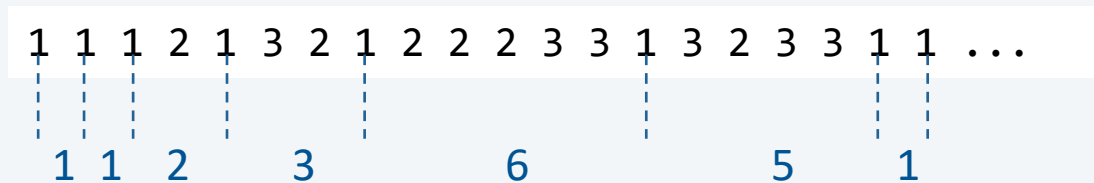
Lückentest

Untersuchung von Regelmäßigkeiten

Wie lange dauert es, bis sich eine Zahl wiederholt?



Beispiel: Lückentest für Wert 1 mit $r = 3$



Auch für großes n kommt nie eine Lücke der Länge 4 vor

Verteilung der Lücken der Länge $i > 1$ ist $p(i) = \left(1 - \frac{1}{r}\right)^{i-1} \cdot \frac{1}{r}$

Chi²-Test (χ^2)

- Chi²-Test ist ein statistischer Test
- Hypothese: Ist-Verteilung f entstammt einer Soll-Verteilung s
- Zufallszahlen in Klassen $1..r$ einteilen
- Häufigkeiten für f_i jede Klasse ermitteln, wobei $f_i > 10$ für alle $1 \leq i \leq r$
- Summe der quadratischen Abweichungen berechnen

$$\chi^2 = \sum_{i=1}^r \left(\frac{(f_i - s_i)^2}{s_i} \right)$$

- Hypothese trifft nach Sedgewick mit hoher Wahrscheinlichkeit zu

$$r - 2\sqrt{r} < \chi^2 < r + 2\sqrt{r}$$

Beispiele

Beispiel ($r = 8, 2.34 < \chi^2 < 13.66$)

i	1	2	3	4	5	6	7	8
s_i	100	100	100	100	100	100	100	100
f_i	85	118	94	81	116	107	88	111
	2.25	3.24	0.36	3.61	2.56	0.49	1.44	1.21

$$\chi^2 = 15.2$$

Wert 15.2 liegt außerhalb
der Abweichung

i	1	2	3	4	5	6	7	8
s_i	100	100	100	100	100	100	100	100
f_i	90	100	90	110	90	100	110	110
	1	0	1	1	1	0	1	1

$$\chi^2 = 6$$

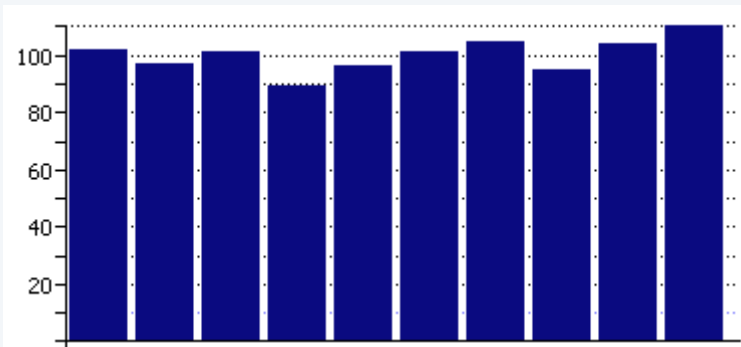
Wert 6 liegt innerhalb der
Abweichung

Visuelle Tests

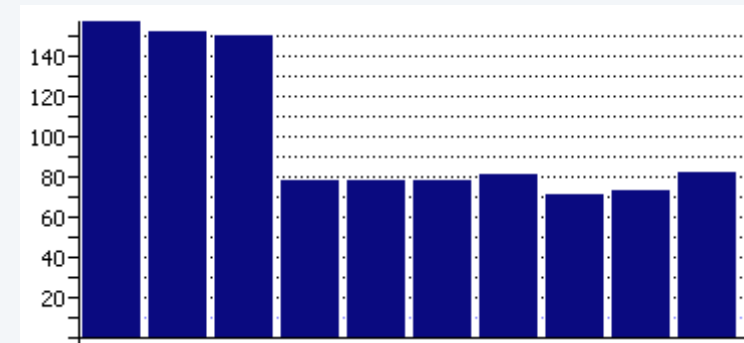
- Grafische Darstellung von erzeugten Zufallszahlen
- Fähigkeit des Auges zur Mustererkennung wird ausgenutzt

Histogramm-Test

- Veranschaulichung des Gleichverteilungstests



Gute Verteilung

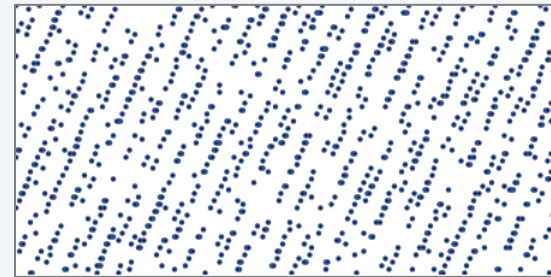
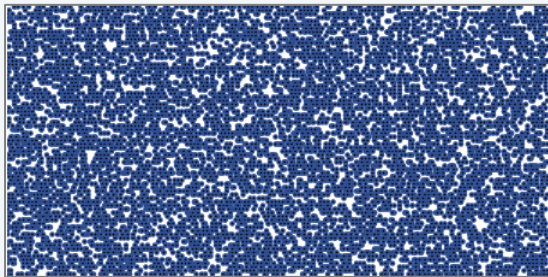
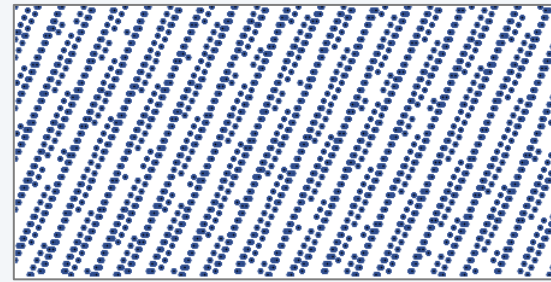
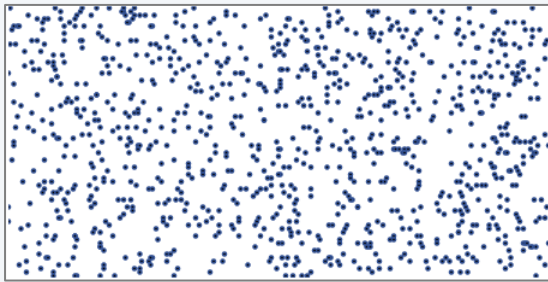


Schlechte Verteilung

- Große Abweichungen und zu flache Verteilung sind gleichermaßen verdächtig

Himmelstest

- Koordinaten x und y (aus Breite/Höhe) für Punkte werden zufällig gewählt und dargestellt
- Muster deuten auf Regelmäßigkeiten hin



Ohne Muster

Mit Muster